Experiment 14: Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst / Custom Functions

Name: Thanashree G R	**Roll No:** 241501262
Department: AIML	**Application:** Patient Appointment Booking System

1. Aim

To explore how a Zoho Creator low-code application can be extended using reusable Deluge custom functions and a Zoho Catalyst serverless function for advanced processing and API-based communication.

2. Introduction and Theory

Zoho Creator provides visual tools for forms, reports, pages and workflows. Deluge custom functions add reusable logic to these components. For tasks that need a separate backend, Zoho Catalyst can host serverless functions written in languages such as Node.js, Java or Python. Creator can communicate with a deployed endpoint using an authenticated API call. This experiment applies the idea to the Patient Appointment Booking System: Creator can count registered appointments locally, while a Catalyst service could later perform more complex processing or return calculated results.

Custom Deluge functions: Reusable scripts that automate application logic and can be tested with Creator's code executor.

Zoho Catalyst: A serverless platform for deploying backend functions and APIs without managing a server.

API integration: A Creator function sends a request to a deployed service and processes the response.

3. Requirements

• Zoho Creator account and the Patient Appointment Booking System application
• Access to Workflow → Functions and Settings → Developer Tools
• Zoho Catalyst account for the optional serverless extension
• Internet connection and a web browser

4. Procedure

Step 1 – Create or open a custom function. Open the application in Zoho Creator. Select **Workflow** → **Functions**. Create a Deluge function or open the existing **CountAppointments** function.

Step 2 – Add and test Deluge logic. Use the function to count records in the Patient Registration form and display the count in the execution log. Save the function and click Execute to test it.

Step 3 – Review developer tools. Open **Settings** and inspect Schema Builder and Application IDE. These tools help view form relationships and the application's underlying structure.

Step 4 – Plan the Catalyst extension. In Zoho Catalyst, create a project and add a serverless function that accepts appointment data and returns a calculated result. Deploy it to obtain an endpoint URL.

Step 5 – Connect Creator to Catalyst. If the account plan and permissions allow it, create an authorized connection in Creator. Call the deployed endpoint from a custom function using invokeUrl/invokeurl and process the returned response.

Step 6 – Validate. Run the Creator function and verify its output. If the Catalyst connection cannot be created because the current trial or plan does not expose Connections, document this part as planned rather than claiming it was executed.

5. Sample Code

A. Deluge function used in the Patient Appointment Booking System

```
void CountAppointments()
{
    totalAppointments = Patient_Registration.count();
    info "Total Appointments: " + totalAppointments;
}
```

The form link name must match the actual form link name in the application. In the current application, the function was executed successfully and the log displayed a total of 8 appointments.

B. Example Creator request to a deployed Catalyst endpoint (template)

```
response = invokeurl
[
    url : "https://<your-catalyst-function-endpoint>"
    type : POST
    parameters : {"recordId": input.ID}
    connection : "catalyst_connection"
];
info response;
```

Replace the endpoint and connection name with the actual values from the deployed Catalyst project. This is a template; it will not run until the endpoint and authorized connection are configured.

6. Expected Result / Observations

The custom Deluge function executes successfully and displays the total number of patient registration records in the log. The Application IDE and Schema Builder are available for inspecting the application. The Catalyst API extension describes how advanced backend processing can be added; its end-to-end execution depends on having a deployed Catalyst endpoint and an available authorized connection.

7. Conclusion

The experiment demonstrates the pro-code extension approach for Zoho Creator using custom Deluge functions and the design of a serverless API integration. The appointment-count function was tested in Creator. The Catalyst integration is documented as a configuration template and should be marked completed only after the endpoint and connection have been configured and tested.

Note: No screenshots are included, as requested. The report uses compact spacing to avoid large empty areas.